מסמך מבוסס קוד

ניסויי L-1 / L1

LayerForge V3

הארכיטקטורה הטכנית מוסברת מן המקור

מדריך מפורט בעברית למודלי הנתונים, עיבוד התמונה, תכנון שכבות חיסורי, אימות מבני, בניית CAD, אלגוריתמי החיפוש והמתמטיקה שממומשת בפועל במאגר.

נבדק מול revision `1491fbb03b6368369147e74690ef128413a3b62a`
בענף `main`
סביבת מקור: Windows, `Python 3.12.1`, קובצי המקור תחת `src/layerforge_l1/`
תאריך בדיקה: 21 באוקטובר 2026

הקדמה, היקף ומתודולוגיית אימות

מסמך זה מתאר את מה שקיים בקוד, ולא ארכיטקטורה רצויה לעתיד. כל טענה מרכזית קושרה לקובץ, לפונקציה ולטווח שורות. מסמכי הפרויקט שימשו להבנת הכוונה והמעמד, אך ההתנהגות הטכנית נקבעה לפי המימוש, הבדיקות ודוחות ההרצה הקיימים.

ממצא מאומת

אין כיום מנוע יחיד שמחבר תמונה שרירותית ישירות ל-CAD דרך CLI אחד. קיימים כמה נתיבים: הוכחת CAD ידנית; מסלול אוטומטי ישן עם מחברים; מתכנני מסכות חיסוריים; CP-SAT ניסויי; ו-`RevealPlanner` חדש יותר שמופעל מסקריפטים.

משמעות

המונח "המתכנן הפעיל" תלוי בנקודת הכניסה. פקודת `auto` של ה-CLI מפעילה את האלגוריתם הישן מבוסס המחברים. `RevealPlanner` הוא המתכנן הניסויי העדכני ביותר שמגובה בארטיפקטים, אך אינו פקודת CLI ציבורית. פקודת `build` בונה תוכנית ידנית מ-manifest.

סימוני אמינות

סימון	משמעות במסמך
ממומש	נראה ישירות בפונקציה פעילה או בסקריפט ביצוע.
הסבר נגזר	נוסחה או אינווריאנט שמתרגמים פעולה קיימת לשפה מתמטית.
רקע תאורטי	הקשר לימודי בלבד; אינו טענה שקוד זה מממש את כל התאוריה.
ניסויי	מודול או סקריפט שקיים ועובד, אך אינו משולב בנתיב המוצר הראשי.
פער	חיבור חסר, ראיה חסרה, או מגבלה מפורשת במימוש.

מצב המאגר בזמן הבדיקה

העבודה נעשתה במצב קריאה בלבד ביחס לקוד. המאגר כבר הכיל שינויים לא-מחויבים ב-`STATUS.md`, `fullres_flatten.py`, `scripts/run_turtle_reveal_planner_poc.py` ו-`reveal_planner.py`, וכן קבצים לא-מנוהלים. הם נחשבו למצב קיים של המשתמש ולא שונו לצורך התיעוד.

תוכן עניינים

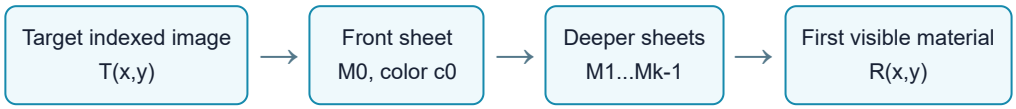
1. מטרת המערכת ומודל המוצר
2. מפת המאגר והארכיטקטורה
3. נקודות כניסה ונתיבי ביצוע
4. קליטת תמונה והכנת נתונים
5. מודלי נתונים פנימיים
6. משפחת אלגוריתמי תכנון השכבות
7. המתמטיקה הממומשת
8. בניית מסכות חומר וחיתוכים
9. אימות חיבוריות ורוחב פיזי
10. נרמול קווי מתאר ותיקון נסתר
11. בניית CAD ויצוא
12. דוגמאות עבודה מלאות
13. בדיקות, ראיות ומגבלות
14. מדריך ניווט למפתח
15. מילון מונחים
16. אינדקס מקור

מפת קריאה מהירה

קורא שרוצה להבין קודם את ליבת התכנון יתחיל בפרקים 6 עד 9. מי שמתחזק יצוא CAD יתחיל בפרקים 8, 10 ו-11. מי שמנסה להבין איזה מנוע רץ בפועל חייב לקרוא את פרק 3 ואת טבלת ההשוואה בפרק 6.

1. מטרת המערכת ומודל המוצר

LayerForge מייצג איור שטוח כערימה מסודרת של יריעות פיזיות. יריעה אינה "אובייקט סמנטי" כמו עין או ענן. היא מסכת חומר בצבע יחיד, בעלת היקף חיצוני משותף, שיכולה לשאת כמה אזורים חזותיים נפרדים. חורים ביריעות קדמיות חושפים צבע מיריעות עמוקות יותר.



העיקרון החיסורי: הצבע המוצג בכל פיקסל הוא צבע היריעה הקדמית ביותר שמכילה חומר באותו מיקום.

IMPLEMENTED MATHEMATICS

$$R(x,y) = c_{\min\{\, d \,:\, M_d(x,y)=1 \,\}}$$

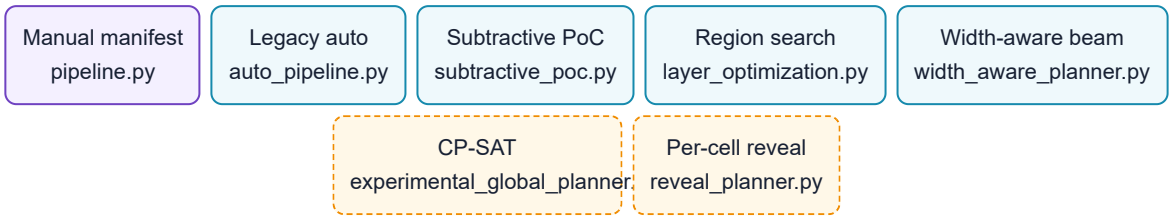
כאן `d=0` הוא העומק הקדמי ביותר, `M_d` היא מסכת החומר הבינרית ו-`c_d` הוא צבע היריעה. המימוש מופיע בצורות שונות ב-`subtractive_poc._compose`, ב-`run_turtle_reveal_planner_poc.run_variant` וב-`hidden_width_repair.compose_back_to_front`.

היקף משותף נוצר כרצועה סביב aperture האמנות. בנתיב הידני הוא פוליגון טבעתי אמיתי; בנתיבי raster הוא מסכה שבה אזור הגבול קבוע ל-`True`. מטרתו רישום XY וחיבור מכני, לא מסגרת חיצונית למוצר.

אינווריאנטים מרכזיים בקוד

- צבע יחיד ליריעה פיזית.
- החומר המצטבר של שכבה קדמית כולל חומר חזותי שלה וחומר נסתר שמותר מאחורי שכבות שכבר מכסות את המקום.
- הקומפוזיט חייב להיות זהה ליעד האינדקסי במסלולים החיסוריים והגלובליים.
- חיבוריות פיזית נבדקת ב-4 שכנים, לא במגע אלכסוני בלבד.
- רוחב מבני נבדק באמצעות erosion של מסכת החומר, ולא באמצעות עובי של חור.

2. מפת המאגר והארכיטקטורה



שבעה מוקדי תכנון/בנייה. הסגול ידני, הכתום ניסויי מבודד. אין ביניהם שרשרת אחת אוטומטית מלאה.

תחום	קבצים עיקריים	תפקיד בפועל
CLI	`cli.py, __main__.py`	פענוח פקודות וייבוא עצל של תת-המערכת המתאימה.
CAD ידני	`pipeline.py, geometry.py, raster.py`	קריאת manifest, Boolean 2D, extrusion, STEP/STL, read-back ו-previews.
אוטומציה ישנה	`auto_pipeline.py`	K-means, שכבה לצבע, חיבור רכיבים בנתיבים נראים, יצוא CAD.
חיסור raster	`subtractive_poc.py`	אינדוקס, קבוצות ספק, מסכות מצטברות, קומפוזיט מדויק.
חיפוש אזורים	`layer_optimization.py, width_aware_planner.py`	גרף רכיבים, bitsets, beam, דירוג, בדיקת גאומטריה אמיתית.
תכנון גלובלי	`experimental_global_planner.py`	CP-SAT בעומק קבוע, זרימה לחיבור prefix, oracle גאומטרי.
Reveal	`reveal_planner.py`	פיצול אזור לתאים, maximal reveal, תמיכה נסתרת וחיפוש רצף צבעים.
אימות ותיקון	`connectivity_repair.py, hidden_width_repair.py`	4-neighbor, erosion core, orphan cores ונתיבי הרחבה נסתרים.
הכנת תמונה	`fullres_flatten.py, contour_normalization.py, contour_consistency.py, tiny_region_cleanup.py`	ניסויי flattening ונרמול קווי מתאר לפני תכנון.

תלויות מדויקות

החבילה דורשת `Python >=3.12,<3.13`. התלויות המוצמדות הן CadQuery 2.6.1, NumPy 2.2.6, OpenCV 4.10.0.84, Pillow 10.3.0, SciPy 1.14.0, Shapely 2.0.6, Trimesh 4.8.3 ו-OR-Tools 9.14.6206. Matplotlib 3.9.0 היא תלות אופציונלית בשם `global-planner`.

pyproject.toml:5-26

3. נקודות כניסה ונתיבי ביצוע

נקודת הכניסה המותקנת היא `layerforge-l1 = layerforge_l1.cli:main`. הקובץ `__main__.py` מפנה לאותה פונקציה. ה-CLI מגדיר פקודות רבות ומבצע imports רק לאחר parsing, כדי שפקודות 2D לא ישלמו את עלות טעינת CadQuery.

src/layerforge_l1/cli.py:33-130, 132-258; pyproject.toml:25-26

פקודה	נתיב אמיתי	פלט עיקרי
`build / verify`	`pipeline.build / pipeline.verify`	CAD ידני של Orbit Bloom ואימות עץ ארטיפקטים.
`auto`	`auto_pipeline.run_auto_case`	הצעה אוטומטית ישנה, CAD ודו"ח התאמה.
`poc2d`	`subtractive_poc.run_poc_case`	מסכות חיסוריות בלבד.
`optimize2d`	`layer_optimization.run_optimization_case`	אופטימיזציית סדר אזורים.
`plan-width-aware2d`	`width_aware_planner.run_width_aware_case`	beam עם oracle רוחב.
`validate-direct2d`	`shadowbox_simplification.run_direct_case`	בדיקת יעד ישיר והפעלת אותו planner.

`experimental_global_planner.main` הוא entrypoint של מודול עצמאי, ו-`RevealPlanner` מופעל דרך סקריפטים כגון `scripts/run_turtle_reveal_planner_poc.py`. לכן הם ממומשים אך לא "ברירת המחדל" של ה-CLI.

> אין פונקציה אחת שמקבלת כיום PNG כללי, בוחרת אוטומטית בין CP-SAT, preprocessing או Reveal, וממשיכה באופן מאומת ל-CAD. יצוא CAD ממסכות Reveal נמצא בסקריפט נפרד.

4. קליטת תמונה והכנת נתונים

4.1 המסלול הישן: quantization גלובלי

`auto_pipeline.quantize_image` קורא RGBA, מרכיב alpha על לבן, משנה גודל וממיר ל-Lab. K-means של OpenCV מייצר מרכזים, התוויות מקנונות בסדר דטרמיניסטי, ורכיבים קטנים מוחלפים לפי טבעת שכנים. רקע נבחר לפי משקל של פיקסלי מסגרת כפול שלוש ועוד שטח; הצבע הכהה ביותר לפי ערוץ L.

src/layerforge_l1/auto_pipeline.py:99-190

IMPLEMENTED MATHEMATICS

alpha composite: $C = \alpha\, C_{src} + (1\text{-}\alpha)\cdot 255$

nearest palette: $\text{label}(p) = \arg\min_j \| \text{Lab}(p) - \mu_j \|^2$

המרחק הוא אוקלידי במרחב Lab של OpenCV. זוהי קירוב פרקטי לדמיון תפיסתי, לא Delta E תקני מלא. K-means ממזער סכום ריבועי מרחקים; הוא אינו מבין עצמים, הצללה או קווי מתאר.

4.2 המסלול החיסורי

`subtractive_poc.analyze_image` מבצע bilateral filter, דגימת פיקסלים, K-means ב-Lab, מיזוג מרכזים קרובים, שיוך nearest-center וניקוי רכיבים זעירים. אחר כך `_provider_regions` מפיק רכיבי neighbor-4 וקבוצות צבע שנגישות מן החוץ.

src/layerforge_l1/subtractive_poc.py:60-199, 202-301

4.3 מתאם lossless

`experimental_global_planner.exact_index` משתמש ב-`np.unique` על שלשות RGB. כל צבע קיים מקבל אינדקס, בלי יצירת צבע ביניים. `load_lossless_target` משתמש ב-nearest-neighbor אם נדרש resize. בבדיקת benchmark נשמרו בדיוק ארבעת הצבעים.

src/layerforge_l1/experimental_global_planner.py:73-122; tests/test_experimental_global_planner.py:24-29

4.4 flattening וניקוי

`fullres_flatten` מבצע bilateral, בחירת palette מפיקסלים שאינם edges, nearest Lab, הצבעת רוב בחלון ומיזוג fragments. `flat_color_workflow.flatten_art_cells` שונה: הוא מזהה גבולות חזקים באמצעות הפרשי Lab, Canny וקו דיו כהה, יוצר תאי אמנות ב-connectivity-4, ומחליף תא רק אם פיזור הצבע שלו נראה חד-שיאי. תאים סמוכים מתמזגים רק כאשר הפרש Lab קטן והתפר אינו נתמך בקו כהה.

src/layerforge_l1/fullres_flatten.py:1-210; src/layerforge_l1/flat_color_workflow.py:93-364

אלה מסלולים ניסויים נפרדים. אין ב-CLI מנגנון שבוחר ביניהם באופן אוטומטי לפי איכות.

5. מודלי נתונים פנימיים

מושג	ייצוג	יוצר	צרכנים
תמונת יעד	`labels: ndarray[H,W]`, `palette_rgb: ndarray[C,3]`	`analyze_image / exact_index`	כל המתכננים
RegionNode	אינדקס, צבע, שטח, bounds, centroid, bits	`build_region_graph`	optimizer, width-aware, CP-SAT
מצב bitset	`int`	`SearchRecord.state`	beam ו-memoization
Provider	קבוצה של region IDs בעומק ובצבע אחד	`_provider_regions / _materialize_analysis`	`construct_layers`
יריעת raster	`physical_mask, cutout_mask, visible_aperture`	`construct_layers / RevealPlanner.materialize`	validator, preview, CAD script
שכבת manifest	JSON עם additive, cutouts, transforms, צבע ועובי	fixture ידני או auto manifest	`pipeline.build`
LayerBuild	spec, Shapely area, CadQuery solid, footprints ו-paths	`pipeline.build`	preview ו-validation

bitsets

רכיב מספר r מיוצג בביט `1<<r`. איחוד קבוצות הוא OR; בדיקת שייכות היא AND; מספר רכיבים הוא `bit_count()`. זה חוסך אובייקטי set ומשמש כמפתח hash דטרמיניסטי.

DERIVED EXPLANATION

$$S' = S \vee G \quad \text{uncovered} = \text{ALL} \wedge \neg S \quad |S| = \text{bit_count}(S)$$

הזהות של מצב ב-`layer_optimization` וב-`width_aware_planner` היא בעיקר קבוצת האזורים שכבר שובצו. נתיבים שונים שמגיעים לאותו bitset נחשבים שקולים לצורך ההמשך, אף שסדר העבר נשמר רק ברשומה הטובה שנבחרה. זוהי הנחת דחיסה חשובה.

6. אלגוריתמי תכנון השכבות שממומשים בפועל

6.1 המתכנן האוטומטי הישן: שכבה לצבע ומחברי MST

`auto_pipeline.propose` מנסה סדרי צבע מוגבלים. לכל צבע, `_shortest_connector_tree` מוצא רכיבי connected components ומרכזים, מוסיף node וירטואלי לקצה, בונה עץ פורש מינימלי במשקל מסלול, ומשרטט קווים ישירים או בצורת L. העלות מעדיפה חומר בצבע עצמו ואז אזורים שכבר מכוסים.

לאחר dilation של המחברים והוספת registration band מתקבלת מסכה פיזית. `_route_order` מרכיב קומפוזיט וסופר mismatch ופיקסלי connector נראים. האלגוריתם מסוגל להפיק CAD, אך המחברים עלולים לשנות את התמונה. הוא אינו planner חיסורי גלובלי.

src/layerforge_l1/auto_pipeline.py:193-337, 339-487, 739-820

6.2 Subtractive PoC: קבוצות ספק greedy

`_provider_regions` מחלץ רכיבי צבע ב-4 שכנים ואת יחסי המגע ביניהם. בכל סבב הוא לוקח את כל האזורים הנגישים של צבע נבחר, מסמן אותם כסופקו, וחוזר. כך אותו צבע יכול להופיע שוב בעומק מאוחר. `discover_order` אינו מחפש סדר חלופי אמיתי; הוא בודק את סדר קבוצות הספק שכבר נגזר.

src/layerforge_l1/subtractive_poc.py:202-410

לכל depth, החומר בתוך aperture הוא איחוד כל האזורים שנמצאים בחזית הנוכחית או עמוק ממנה לפי מיפוי provider. ההיקף מתווסף תמיד. החורים הם המשלים בתוך aperture.

6.3 אופטימיזציית Region Bitset

`build_region_graph` יוצר node לכל רכיב bits-4, neighbor, adjacency bitsets, לפי צבע ו-border bits. `optimize_region_sequence` מחזיק beam של `SearchRecord`. פעולה חוקית בוחרת צבע ולוקחת את כל האזורים של אותו צבע שנגישים במצב הנוכחי. הדירוג כולל מספר שכבות, חזרות צבע, חומר נשמר וגבול תחתון לפי צבעים שנותרו.

src/layerforge_l1/layer_optimization.py:55-72, 98-358

> החיפוש מוגבל ב-`max_states` וב-beam, ולכן תוצאה היא הטובה שנמצאה תחת התקציב. הוא אינו הוכחת optimum.

6.4 Width-aware beam

המודול מוסיף evaluator של המסכה הפיזית האמיתית לכל `_candidate_groups`. bitset כולל קבוצות legacy וגם cohorts בעלי חתימת topology משותפת: supports, מגע בגבול וצבעי עתיד. כך subsets של אותו צבע יכולים לחלוק יריעה.

הדירוג החדש בוחן המשכים חוקיים, סכנת dead-end, אזורים וצבעים נגישים, orphan cores, שכבות וחזרות צבע. בחירת beam שומרת מחצית מהקיבולת לחתימות המשך שונות ואז ממקסמת מרחק Jaccard בין assigned sets. כל transition עדיין מקומי ובלתי הפיך.

src/layerforge_l1/width_aware_planner.py:99-161, 163-333, 335-519, 532-853

6.5 CP-SAT גלובלי בעומק קבוע

זהו backend מבודד. לכל region ולעומק יש boolean `x[r,d]`; לכל עומק וצבע `y[d,c]`; ו-`p[r,d]` מציין אם region כבר מופיע ב-prefix עד עומק d. כל region משויך בדיוק לעומק אחד וכל שכבה בדיוק לצבע אחד ואינה ריקה. התאמת צבע נאכפת על ידי `x[r,d] ≤ y[d,color(r)]`.

חיבוריות prefix נאכפת בקירוב טופולוגי מדויק לגרף באמצעות single-commodity flow מן perimeter root. רוחב raster אינו ב-CP-SAT; הוא oracle חיצוני. הצעה שנכשלת מוסיפה exact-prefix no-good בלבד, כלומר נאסר רק אותו bitset בעומק הכושל.

src/layerforge_l1/experimental_global_planner.py:131-309, 334-405, 438-742

> Connected source-color region אינו יכול להתפצל בין עומקים. מספר השכבות K קבוע בכל ריצה וכל שכבה חייבת להיות לא ריקה. לכן INFEASIBLE מוכיח רק חוסר פתרון במודל המוגבל הזה.

6.6 RevealPlanner: תכנון reveal לפי תאים

זהו backend הניסויי החדש ביותר שמגובה בדוחות הנוכחיים. הוא מפצל region צבעוני לתאים לפי grid פיזי, ומאחד fragments קטנים לאותו source region. מצב הוא boolean vector של תאים שכבר מכוסים מלפנים. עבור כל צבע, `maximal_reveal` מתחיל בכל התאים הבלתי מכוסים של הצבע ומסיר שוב ושוב תאים שיש בהם פיקסלים לא נתמכים עד נקודת שבת.

`structural_support` שוחק את תחום ה-covered, שומר cores המחוברים להיקף, מרחיב אותם חזרה, ומנסה לתקן מגעים אלכסוניים דרך footprints שנשארים בתוך covered. תאים גלויים מצורפים לאחר מכן. מכיוון שכל חומר תמיכה נמצא מאחורי משהו שכבר כוסה, הוא אינו משנה את הקומפוזיט.

`beam_search` בוחר רצף צבעים קצר של maximal reveals, deduplicates states באמצעות `np.packbits`, ומדרג לפי שטח מכוסה או מספר צבעים לא גמורים. `eliminate_sheets` מנסה להעביר reveal לשכבה מאוחרת באותו צבע ורק אם כל שכבות הביניים נשארות חוקיות.

src/layerforge_l1/reveal_planner.py:37-158, 171-234, 237-460

השוואה מדויקת

מנוע	יחידת החלטה	גלובלי?	רוחב בזמן תכנון	סטטוס שילוב
Legacy auto	צבע + components	לא	לא	CLI auto
Subtractive greedy	provider regions	לא	אחרי בנייה	CLI poc2d
Region optimizer	כל reachable region בצבע	beam מוגבל	בעיקר topology	CLI optimize2d
Width-aware	קבוצת regions	beam מוגבל	כן, actual mask	CLI
CP-SAT	region שלם לעומק	joint fixed-K	oracle איטרטיבי	מודול עצמאי
Reveal	תא בתוך region	beam על רצף צבעים	כן, construction-aware	סקריפטים ניסויים

7. יסודות מתמטיים של המימוש

7.1 המרת יחידות ומורפולוגיה

IMPLEMENTED MATHEMATICS

$$m_{px} = S_{mm} / N_{outer}$$

$$w_{enforced} = w_{nominal} + w_{vector} + w_{CAD}$$

$$q = \text{ceil}(w_{enforced} / m_{px})$$

$$k = q \text{ if } q \text{ odd, else } q+1$$

m_px הוא מילימטר לפיקסל, S_mm גודל היריעה, ו-N_outer מספר הפיקסלים כולל perimeter. ערכי ברירת המחדל הם 0.8 + 0.2 + 0.2 = 1.2 מ"מ. kernel אליפטי חייב להיות אי-זוגי כדי שיהיה לו פיקסל מרכז.

src/layerforge_l1/connectivity_repair.py:114-149

דוגמה מספרית. ביריעה 200 מ"מ וברשת 2304 פיקסלים, m_px=0.086806 mm/px. דרישת 1.2 מ"מ היא 13.824 פיקסלים; ceiling נותן 14 ולכן kernel ממורכז הוא 15x15. הקוד בודק footprint דיסקרטי של 15 פיקסלים, כלומר מעט שמרני יותר מן הערך הרציף.

erosion core 7.2

IMPLEMENTED MATHEMATICS

$$\text{Core} = M \ominus K$$

$$\text{Anchored} = CC_4(\text{Core}) \text{ that intersects Perimeter}$$

$$\text{Orphan} = \text{Core} \setminus \text{Anchored}$$

ה-erosion משאיר רק מרכזים שבהם footprint K נכנס כולו בחומר. אם core משמעותי אינו מחובר ל-core המעוגן בהיקף, לפחות neck אחד נעלם תחת דרישת הרוחב. PASS מתקבל רק כאשר אין orphan cores. רכיב גלוי שלא שורד erosion מסווג כ-thin decorative warning ולא ככשל load-bearing.

src/layerforge_l1/connectivity_repair.py:152-165, 227-350

7.3 חיבוריות 4 שכנים

גרף raster כולל קשתות רק בין פיקסלים שחולקים צלע. שני פיקסלים שנוגעים בפינה נמצאים ברכיבים שונים. OpenCV מופעל עם connectivity=4. הבדיקה מדווחת גם neighbor-8 להשוואה, אך PASS תלוי ברכיב neighbor-4 יחיד.

IoU 7.4 והתאמת projection

IMPLEMENTED MATHEMATICS

$$IoU(A,B) = |A \cap B| / |A \cup B|$$

במסלול CAD הידני raster של המקור מושווה ל-footprints שנקראו מחדש מן STL. נמדדים IoU של silhouette ושל כל צבע, coverage ומספר mismatch pixels. זו התאמת projection, לא הוכחה לטולרנס מכני או לחוזק.

src/layerforge_l1/pipeline.py:257-315

CP-SAT flow 7.5

IMPLEMENTED MATHEMATICS

$\Sigma_d x_{r,d}=1;\ \ \Sigma_c y_{d,c}=1$

$p_{r,d}=\Sigma_{i=0..d} x_{r,i}$

$\Sigma\ \text{incoming}(r)-\Sigma\ \text{outgoing}(r)=p_{r,d}$

$\min \Sigma_{r,d}\ \text{area}(r)(K-d)x_{r,d}$

הזרימה דורשת יחידת flow לכל region פעיל ב-prefix. ה-root שולח את סכום היחידות. קיבולת edge מוגבלת לאפס אם אחד מקצותיה אינו ב-prefix. פונקציית המטרה מעניש region שמופיע מוקדם משום שהוא נשמר כחומר נסתר במספר רב יותר של prefixes.

7.6 נקודת שבת של maximal reveal

DERIVED EXPLANATION

$C_0 = \{\text{cells of color c not covered}\}$

$C_{i+1} = C_i \setminus \text{Unsupported}(\text{covered}, C_i)$

$\text{stop when } C_{i+1}=C_i$

המועמד רק קטן, ולכן הלולאה מסתיימת אחרי לכל היותר מספר התאים של הצבע. התוצאה היא נקודת שבת של פונקציית הסרה הספציפית למימוש. אין כאן הוכחה כללית שהיא הקבוצה הגדולה ביותר בכל מודל גאומטרי אפשרי, מפני ש-`structural_support` עצמו heuristic raster מוגבל.

8. מסכות פיזיות, חומר וחיתוכים

8.1 Boolean CAD ידני

IMPLEMENTED MATHEMATICS

$$\text{LayerArea} = \text{union}(\text{AdditiveRegions}) \setminus \text{union}(\text{CutoutRegions})$$

`geometry.layer_area` ממשם union באמצעות Shapely, difference ואז `buffer(0)` לתיקון topology. Hole נשמר כ-interior ring של `extrude_geometry`. Polygon יוצר CadQuery Face מן exterior וה-holes ומבצע extrusion.

src/layerforge_l1/geometry.py:29-109

8.2 מסכה מצטברת חיסורית

ב-Subtractive PoC, CP-SAT, region-ובבעל depth $\delta(r)$ שייך לחומר של prefix d אם $\delta(r) \le d$. הוא נראה בשכבה d אם $\delta(r) = d$. ההיקף מתווסף מחוץ ל-aperture.

IMPLEMENTED MATHEMATICS

$$\text{Retained}_d(x,y) = [\delta(\text{region}(x,y)) \le d]$$

$$\text{Visible}_d(x,y) = [\delta(\text{region}(x,y)) = d]$$

$$\text{Physical}_d = \text{Perimeter} \cup \text{Retained}_d$$

$$\text{Cutout}_d = \text{Aperture} \setminus \text{Retained}_d$$

src/layerforge_l1/subtractive_poc.py:412-465; src/layerforge_l1/experimental_global_planner.py:334-347

Reveal material 8.3

ב-RevealPlanner, החומר אינו כל `structural_support` covered pixels. בוחר subset רחב ומעוגן של covered domain; אחר כך union עם visible. זו הבחנה חשובה: המתכנן רשאי להשמיט חומר נסתר שאינו נחוץ.

> האלגוריתם אינו משנה צבע גלוי: support מותר רק בתחום שכבר מכוסה מלפנים. אם פיקסל support לא נמצא בתחום הזה, הוא עלול להיחשף ולכן אינו חוקי.

8.4 קוטביות מסכות

בכל מסכות retained שנשמרות, לבן/255 מייצג חומר ו-0 מייצג ריק. cutout הוא המשלים בתוך ה-aperture. Preview צבעוני מצייר את צבע הפילמנט היכן שהמסכה true ולבן היכן שאין חומר. אין inversion ב-`run_turtle_reveal_planner_poc`.

9. אימות פיזי וחזותי

9.1 shared perimeter

ב-CAD הידני נבדקים bounds, Hausdorff distance בין outer rings, שטח perimeter חסר ושטח מחוץ למעטפת. ב-raster נבדק שכל פיקסל ב-perimeter mask הוא חומר. אלה שתי בדיקות שונות לאותו אינווריאנט.

src/layerforge_l1/geometry.py:148-211; scripts/run_turtle_reveal_planner_poc.py:128-139

9.2 חיבוריות

`audit_edge_connectivity` מפיק labels נפרדים ב-4 וב-8 שכנים, מזהה אילו רכיבי 4 נוגעים בגבול, וסופר orphan components. הסטטוס PASS דורש בדיוק רכיב neighbor-4 אחד בכל מסכת החומר.

9.3 רוחב load-bearing

לאחר erosion, כל core שאינו מחובר ל-perimeter הוא כשל. `_measure_orphan_bottlenecks` מריץ kernels אי-זוגיים מ-1 עד הדרישה ומדווח את ה-kernel הגדול האחרון שבו היעד עדיין נגע ב-core מחובר. זהו מדד דיסקרטי, לא מדידת neck רציפה.

9.4 מול decorative structural

רכיב גלוי ללא חפיפה ל-core מסווג כ-`thin_decorative_visible`. רכיב גלוי שחופף orphan core מסווג structural bottleneck. PASS אינו נכשל בגלל warning דקורטיבי בלבד. Narrow openings אינם עוברים כלל דרך בדיקת retained width.

> בכמה סקריפטים קיימת בדיקת פתחים נפרדת באמצעות morphological opening וסף 0.8 מ"מ, אך היא diagnostic בלבד ואינה חלק מ-PASS של 1.2 מ"מ.

9.5 דיוק קומפוזיט

הקומפוזיט נבנה מן המסכות עצמן. במסלולים החיסוריים נדרש `geometry_mismatched_pixels == 0` וגם `visible_automatic_connector_pixels == 0`. המסלול האוטומטי הישן מדווח mismatch אך עשוי להגיע ל-READY_FOR_USER_TEST גם כאשר הפיקסלים אינם זהים, כל עוד בדיקות היריעות עברו; זו סמנטיקה שונה.

מה PASS אינו מוכיח

- אין סימולציית עומסים, כיפוף או שבר.
- אין הוכחת printability של כל חור צר.

- אין פיצוי shrinkage של חומר או טולרנס מדפסת.
- אין הוכחת minimum global layer count.
- אין בדיקת frame או mounting, שהם מחוץ להיקף.

10. נרמול קווי מתאר ותיקון רוחב נסתר

10.1 Zhang-Suen ו-tracing

`contour_normalization.zhang_suen_thinning` מבצע שתי תת-איטרציות של thinning. פיקסל מוסר רק אם מספר השכנים ומספר מעברי 0→1 סביבו עומדים בתנאים, ובכל תת-איטרציה נשמרים חיבורים בכיוונים מסוימים. התוצאה היא skeleton בעובי פיקסל.

`trace_skeleton_segments` הופך את ה-skeleton לגרף, מסיר shortcut אלכסוני כאשר קיים צעד אורתוגונלי, ומפרק strokes בנקודות degree שאינו 2. רוחב אפקטיבי נמדד מ-distance transform ב-resolution מוגדל.

src/layerforge_l1/contour_normalization.py:73-242

10.2 restroking

לכל path נבחר target width לפי maximum בין 1.2 מ"מ לבין רוחב נומינלי נגזר. path מצויר מחדש ברזולוציה גבוהה, מורד ל-raster דיסקרטי, ונפסל אם הוא מחבר component כהה זר או משנה component counts של צבעים שאינם contour. `contour_consistency` מבצע restroking מלא של path כדי למנוע blobs מקומיים.

10.3 תיקון חומר נסתר

`hidden_width_repair.hidden_addition_domain` מגדיר היכן מותר להוסיף חומר: רק איחוד מסכות קדמיות שמסתירות את התוספת. `_shortest_existing_material_paths` מבצע BFS ב-4 שכנים בתוך חומר קיים בין core מחובר לבין orphan goals. הנתיב מעובה ב-kernel, אבל רק בתוך domain המותר.

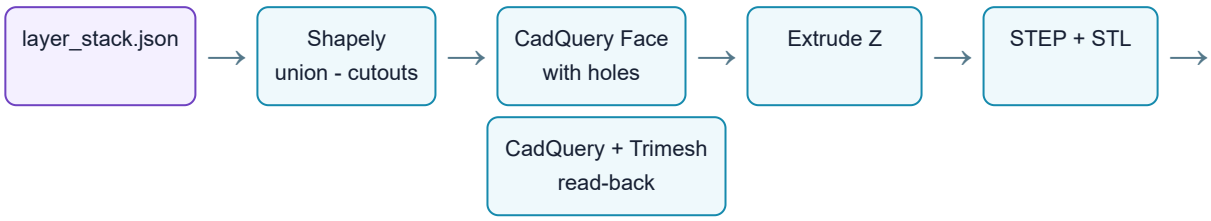
src/layerforge_l1/hidden_width_repair.py:36-274

> בדיקת regression סינתטית מאמתת שאפשר לתקן neck נסתר בלי לשנות את הקומפוזיט; בדיקה נוספת מאמתת שהאלגוריתם מסרב להוסיף חומר כאשר domain ריק.

tests/test_hidden_width_repair.py:41-90

11. בניית CAD ויצוא

11.1 מסלול manifest הידני



הנתיב היציב של L-1: מקור ידני, Boolean אמיתי, solids וקריאה חוזרת.

`pipeline.build` קורא manifest, בונה לכל layer את Shapely area ואת CadQuery solid, מייצא STEP ו-STL, פותח מחדש, ובודק validity, solid count, watertightness, component count, bounds, volume, bed fit ו-footprint.

src/layerforge_l1/pipeline.py:407-675; src/layerforge_l1/geometry.py:91-130

Assembly transform ו-print transform נשמרים בנפרד. ה-solid עצמו נבנה בקואורדינטות model; placement להדפסה אינו משנה את גאומטריית ההרכבה. Rim אופציונלי נבנה כהפרש בין האזור לבין erosion פנימי ומקבל extrusion נוסף.

11.2 CAD ממסכות Reveal

הסקריפט `export_reveal_stack_cad.py` מוצא contours עם hierarchy מסוג CCOMP, ממיר מרכזי פיקסלים למ"מ, בונה Polygon עם holes, מבצע buffer של חצי פיקסל כדי לעבור מקווי מרכז לקצוות פיקסל, simplify טופולוגי, extrusion ויצוא.

IMPLEMENTED MATHEMATICS

$$x_{mm} = (x + 0.5)m_{px}$$

$$y_{mm} = (H - y - 0.5)m_{px}$$

$$G = \mathrm{simplify}(\mathrm{buffer}(\mathrm{union}(\mathrm{polygons}),\ 0.5m_{px}),\ \varepsilon)$$

scripts/export_reveal_stack_cad.py:29-50, 53-119

STEP נפתח ב-CadQuery ונבדק מספר solids. STL נפתח ב-Trimesh ונבדקים bounds, components, watertightness, משולשים ונפח מול `thickness × area`. הסקריפט כותב סדר assembly back-to-front ו-`assembled_z0_mm=i·thickness`, בעוד כל STL מיועד ל-`z=0` להדפסה.

> יצוא Reveal אינו מודול production משולב. הוא סקריפט נפרד, אינו יוצר cutout-mask CAD נפרד, ואינו מחובר ל-CLI. התאמת raster למסכה נבדקה בארטיפקטים, אך אין schema מאוחד עם manifest הידני.

12. דוגמאות עבודה

דוגמה A: מטריצת מסכות קטנה

ניקח יעד 5x5 עם רקע כחול B, פנים צהובות Y ושני פרטים שחורים K. המטרה היא לא ליצור "שתי עיניים שחורות" כחלקים נפרדים, אלא לחשוף יריעה שחורה דרך חורים בצהוב.

```
FRONT Y MASK
0 0 0 0 0
0 1 0 1 0
0 1 1 1 0
0 1 0 1 0
0 0 0 0 0

BACK K MASK
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

```
TARGET
B B B B B
B Y K Y B
B Y Y Y B
B Y K Y B
B B B B B
```

בקומפוזיט front-to-back, פיקסל Y מוצג במקום שבו מסכת Y היא 1. בשני החורים מסכת Y היא 0 ולכן K העמוק נראה. אין צורך לחבר שתי נקודות K זו לזו: יריעת K רציפה ממילא. זהו בדיוק עקרון `_compose` ו-`RevealPlanner.materialize`.

דוגמה B: benchmark אמיתי של ארבעה צבעים



דוגמת יריעת reference לצורך אימות בלבד; אינה קלט למתכנן.



הקומפוזיט היחיד שנמסר למתכנן.

המתאם lossless שומר 512x512 וארבעה צבעים: graph. #FAF7F0, #F2C84B, #ED852B, #242939 מכיל 19 regions; ספירת regions לפי צבע היא 9, 5, 3, 2; ורק region אחד נוגע בגבול מודל CP-SAT בן חמש שכבות יוצר 95 משתני x ו-20 משתני y, בנוסף prefixes, depth variables ו-flow.

tests/test_experimental_global_planner.py:24-58

ה-reference הידני כולל חמש יריעות, כאשר לבן חוזר בשני עומקים. fixture report מדווח neighbor component-4 יחיד, אפס orphan cores ו-zero roundtrip mismatch לכל היריעות, אך גם מציין במפורש שהוא היה provisional ונדרש אימות LayerForge עצמאי. בדיקות הקוד מאמתות שהמתכנן אינו קורא את תיקיית `private_reference_masks`.

דוגמה C: RevealPlanner על turtle

דוח `l1_turtle_reveal_planner_poc_v1` מתעד 4,005 תאים, closure אפשרי ב-19 סבבים, חיפוש COMPLETE ב-825 הערכות, 18 יריעות, אפס mismatch ב-x4 וכל היריעות PASS ב-neighbor-4 וברוחב. זמן כולל היה 864.7 שניות. גרסת full-resolution מאוחרת יותר תיעדה 3,957 תאים ו-16 יריעות; גרסת tile 2.0 מ"מ תיעדה 12,011 תאים ו-15 יריעות.

אלה ראיות ביצוע קיימות, לא ריצה שבוצעה לצורך המסמך. הן גם ממחישות trade-off: פיצול עדין יותר מגדיל מספר תאים ועלות חישוב, אך יכול להפחית seams ושכבות.

דוגמה D: CAD קיים ממסכות Reveal

בארטיפקט Van Gogh option B קיימות 10 יריעות CAD. manifest מדווח STEP עם solid יחיד בכל שכבה, STL watertight ורכיב יחיד, bounds של 225x225x1.6 מ"מ. סטיית traced-area המרבית מול מסכה היא 0.085%. דו"ח revalidation מדווח PASS, `px_diff_from_mask=0` בחיבוריות וברוחב לכל 10 היריעות, ו-`stack_mismatch_px=0`.

> האיור הוא נכס משתמש ולא משמש כאן כהוכחת זכויות לפרסום. המסמך מציג רק מדדים; אין להעתיקו למאגר ציבורי ללא רישיון.

13. בדיקות, סטטוס ומגבלות

מפת בדיקות

קובץ בדיקה	מה הוא מגן
`test_l1_feasibility.py`	6 שכבות Orbit Bloom, STEP/STL read-back, bounds, holes, previews, transforms והיקף משותף.
`test_subtractive_poc.py`	קומפוזיט מדויק, חורים חושפי צבע, repeated depths ואי-הוספת bridges.
`test_width_aware_planner.py`	actual-mask width gate, 4-neighbor, bear regression וכשלים מפורשים.
`test_experimental_global_planner.py`	lossless palette, 19-region model, אי-שימוש במסכות פרטיות.
`test_reveal_planner.py`	מקרה שבו region indivisible נתקע אך per-cell מוצא stack מדויק של 6 שכבות.
`test_contour_normalization.py`	קווים ישרים, עקומים, אלכסוניים וצמתים.
`test_hidden_width_repair.py`	תיקון נסתר ששומר קומפוזיט וסירוב לשינוי גלוי לא מורשה.

סטטוס לפי MILESTONES

L-1 מוגדר עדיין READY_FOR_USER_TEST או AWAITING_PHYSICAL_TEST בהתאם לארטיפקט. קיימת הוכחת CAD ידנית מלאה, אך אישור הדפסה פיזית ממתין. עבודות L1 האוטומטיות הן ניסויים ואינן מאושרות כמוצר. אין בסיס להצהיר על מעבר ל-L0 או milestone מאוחר.

מגבלות מוכחות

- המסלול האוטומטי הישן יוצר bridges נראים בדוגמאות מורכבות.
- המתכנן greedy עלול להפיק הרבה repeated-color depths.
- width-aware beam יכול להתכנס ל-dead end תחת beam קטן.
- CP-SAT אינו מפצל connected region בין עומקים.
- RevealPlanner יכול לפצל region, אך הוא heuristic, יקר ומייצר seams פוטנציאליים.
- פתחים צרים מדווחים בנפרד ואין gate מוצר אחיד עבורם.
- CAD מ-Reveal קיים בסקריפט נפרד, לא ב-pipeline האוטומטי המאוחד.

פערים שלא אומתו

לא בוצעה ריצת optimization חדשה למסמך. לכן אין טענה שהארטיפקט העדכני הוא optimum, שאין regression לאחר השינויים הלא-מחויבים, או שכל הסקריפטים עדיין משחזרים byte-for-byte את הדוחות. לא בוצעה הדפסה פיזית.

14. מדריך ניווט למפתח

שאלה	התחל כאן	המשך לכאן
איך בנוי solid?	`geometry.build_layer_solid`	`pipeline._export_layer`
איך חור חושף צבע?	`subtractive_poc.construct_layers`	`_compose`
איך נמדדת חיבוריות?	`connectivity_repair.audit_edge_connectivity`	`audit_connection_width`
איך CP-SAT מחבר prefix?	`build_fixed_depth_model`	`_validate_assignment_geometry`
איך Reveal מוצא שכבה?	`maximal_reveal`	`structural_support, beam_search`
מה באמת מופעל ב-CLI?	`cli.parser`	`cli.main`
איך CAD נקרא מחדש?	`pipeline._load_step, _load_stl`	`pipeline.build`

כללי שינוי בטוח שנגזרים מן הקוד

1. אל תשנה קוטביות מסכה בלי לעדכן preview, compose ו-validator יחד.
2. אל תחליף neighbor-4 ב-8-contact; corner neighbor אינו חיבור ברוחב חיובי.
3. שמור הפרדה בין retained width לבין opening width.
4. אם משנים resolution, חשב מחדש `mm_per_pixel` ואת kernel האי-זוגי.
5. ב-CP-SAT, no-good חייב להישאר sound וממוקד ב-prefix המדויק.
6. ב-Reveal, חומר תמיכה חדש חייב להיות subset של covered domain.
7. STEP/STL ו-previews חייבים להיגזר מאותה גאומטריה.

15. מילון מתמטי וטכני

מונח	הגדרה בהקשר LayerForge
aperture	המלבן הפנימי שבו נמצאת האמנות, ללא היקף הרישום.
retained material	פיקסלים/שטח שנשארים כחומר מוצק ביריעה.
through-cutout	חור אמיתי ביריעה שמאפשר לראות יריעה עמוקה יותר.
prefix	קבוצת כל האזורים שכבר הופיעו מן החזית עד depth נתון.
hidden support	חומר שמחבר מבנה אך מוסתר על ידי שכבה קדמית.
structural core	החומר ששורד erosion של footprint דרישת הרוחב.
orphan core	core שאינו מחובר ב-4 שכנים ל-core של ההיקף.
maximal reveal	נקודת שבת של קבוצת תאים מאותו צבע שניתנים לחשיפה יחד.
no-good cut	אילוץ שמונע חזרה על assignment גאומטרי שנדחה.
region seam	גבול בתוך source region כאשר תאים שלו שובצו לעומקים שונים.
watertight mesh	סגור טופולוגית; תנאי נדרש אך לא מספיק להדפסה מוצלחת.

מערכות צירים

תמונת raster משתמשת בראשית בפינה שמאלית-עליונה ו-y כלפי מטה. מודל CAD משתמש בראשית תחתונה-שמאלית ו-y כלפי מעלה. `raster._pixel_ring` ו-`export_reveal_stack_cad.to_mm` מבצעים היפוך y פעם אחת. Z של assembly גדל לפי סדר back-to-front; קובצי print עצמם נשארים על Z=0.

16. אינדקס מקור מאומת

קונספט	קובץ ופונקציה	שורות
CLI dispatch	cli.py: parser, main	33-258
Boolean layer	geometry.py: layer_area	62-69
Extrusion with holes	geometry.py: _face, extrude_geometry	91-109
Shared outline	geometry.py: validate_shared_area	148-189
Manual CAD pipeline	pipeline.py: build	407-675
Legacy connector planner	auto_pipeline.py: _shortest_connector_tree, _route_order	213-318
Subtractive preprocessing	subtractive_poc.py: analyze_image	137-199
Provider grouping	subtractive_poc.py: _provider_regions	202-301
Mask construction	subtractive_poc.py: construct_layers	420-465
Region graph	layer_optimization.py: build_region_graph	98-204
Bitset search	layer_optimization.py: optimize_region_sequence	222-358
Width-aware candidates	width_aware_planner.py: _candidate_groups	163-333
Diverse beam	width_aware_planner.py: _select_diverse_beam, _search	459-806
neighbor audit-4	connectivity_repair.py: audit_edge_connectivity	82-111
Width conversion	connectivity_repair.py: _width_parameters	114-149
Orphan-core audit	connectivity_repair.py: audit_connection_width	227-350
CP-SAT model	experimental_global_planner.py: build_fixed_depth_model	166-309
Geometry oracle	experimental_global_planner.py: run_checkpoint	438-742
Reveal cells	reveal_planner.py: build_cells	85-158
Hidden support	reveal_planner.py: structural_support	171-208
Maximal reveal	reveal_planner.py: maximal_reveal	290-304
Reveal beam	reveal_planner.py: beam_search	374-421
Contour skeleton	contour_normalization.py: zhang_suen_thinning	73-121
Hidden repair	hidden_width_repair.py: repair_one_mask	165-275
Mask-to-CAD	scripts/export_reveal_stack_cad.py: mask_to_geometry	29-50

סיכום מצב אמיתי

LayerForge כבר מכיל ליבה משמעותית: Boolean geometry, CAD solids אמיתיים, מסכות חיסוריות, אימות raster פיזי, כמה אסטרטגיות חיפוש וראיות ל-stacks מדויקים. עם זאת, הארכיטקטורה עדיין מחקרית ורב-מסלולית. הוכחת CAD הידנית יציבה; מנוע התמונה-לשכבות עדיין אינו נתיב מוצר מאוחד; CP-SAT ו-Reveal הם ניסויים בעלי מגבלות שונות; ואישור פיזי נותר ממתין.

> **מסקנת ניווט:** לצורך הבנת ההתנהגות העדכנית ביותר של תכנון פיזי יש לקרוא את `reveal_planner.py` יחד עם סקריפטי ההרצה. לצורך הבנת ה-CLI הציבורי יש להתחיל ב-`cli.py`. לצורך CAD המאומת ביותר יש להתחיל ב-`pipeline.py` וב-`geometry.py`.